

Concept

The previous project focused on authenticated data submitted to the Log Analytics Data Ingestion API 2.0 using a service principal. While that was fairly secure, it had a weakness in that the private keys deployed to all lab devices were identical. This can be addressed by deploying device certificates using SCEP. This certificate can be used to encrypt the connection between device and data submission endpoint via Mutual TLS (mTLS).

This allows us to move the authentication flow to the cloud, no longer requiring the JWT assertion to obtain an access token, reducing complexity of the inventory scripts.

mTLS only requires a certificate, all validation that the correct certificate is being used must be done in the Azure function. This requires a PEM of each certificate in the trust chain (Intermediate and Root CAs) be stored somewhere, and the most secure method is an Azure Key Vault.

This lab is to simulate and document this environment.

Certificates

SCEP generates subordinate certificates signed by a certificate authority (CA), in our case the extended key usage (EKU) properties of the subordinate must enable the private key to be used for the “ssl-client” purpose. We also have to simulate the trust chain, so we need a Root CA and any Intermediate CA’s public keys that can be used to verify properly signed these subordinate certificates.

The subordinate certificate CN is populated using a template in SCEP, so the best way for our inventory scripts to detect which certificate to use is a naming convention like “CN={{DeviceId}}, OU=<Identifier>” and then search all certificates to find the matching Identifier in the subject.

Get-Cert.ps1

To make the process of creating certificate files in the needed format, the Get-Cert Powershell script can automate the process in Windows for a lab without SCEP.

All the configuration variables are in the first block of code.

Update these with your values and run the script in the PowerShell_ISE, you do not need to be an administrator.

<RootFriendlyName> - The full name for the root certificate. (Ie. "Lab_Root_CA2")

<FriendlyName> - The full name for your certificate. (Ie. "AF_Collect")

<MyPassword> - The password required to access the private keys for both certificates.

<DNSName> - What you use for a domain name.

Make sure you have a record of the password, friendly name and certificate thumbprint for each certificate.

The certificates will be in your

"Documents" folder. Make sure you save these files somewhere secure.

```
Get-Cert.ps1 X
1 # -----
2 # PARAMETERS FOR SELF-SIGNED CERT
3 # -----
4 $RootFriendlyName = "<Root Cert Name>"
5 $FriendlyName = "<Leaf Cert Name>"
6 $MyPassword = "<Your Password>"
7 $DNSName = "<Your domain>"
8 #
9 cls
10 Write-Host "Certificate Generation begin."
```

```
Certificate Generation begin.
Root CA Thumbprint: ac463065732e5f0017b425cffce05a99e01a4308
Root CA Friendly Name: IntuneLab_ROOT_CA
Root CA Subject: CN=IntuneLab_ROOT_CA

Certificate Thumbprint: 18b6efa15ccf580fdf497619328d4a9c259851a5
Friendly Name: Data Collect
Subject: CN=Data_Collect
Certificate Generation complete.
```

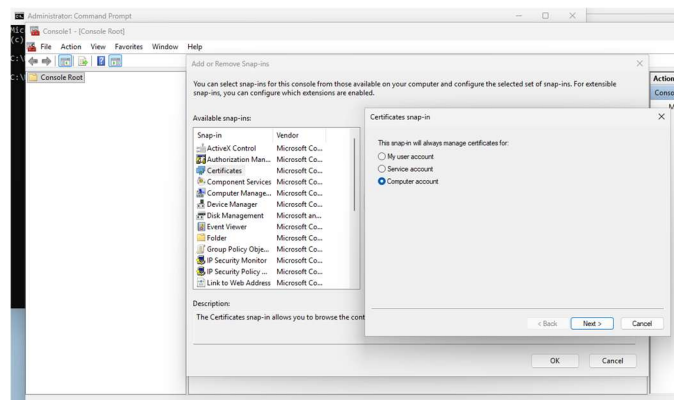
Installing the certificates manually

Windows Certificate

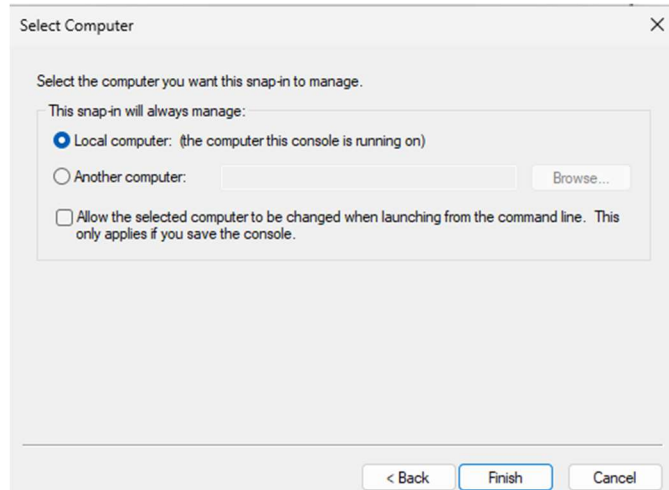
Since this is a test machine, you will be running the script as a user. The LocalMachine\My certificate store restricts users from reading the private key information from the certificate. You need to have administrative rights to perform this step.

Use the Windows search to find the icon for "CMD". Right-click on the icon and choose "Run as Administrator". Using the command prompt, run "MMC.EXE", Select "File", "Add/Remove Snapin..." and add the "Certificates" snap in.

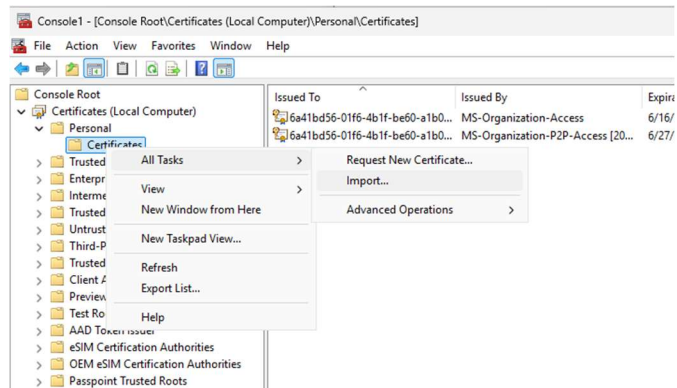
Select "Computer Account" and click "Next."



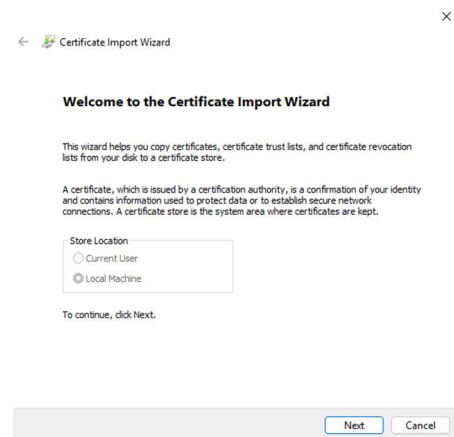
Select “Local Computer” and click “Finish”.



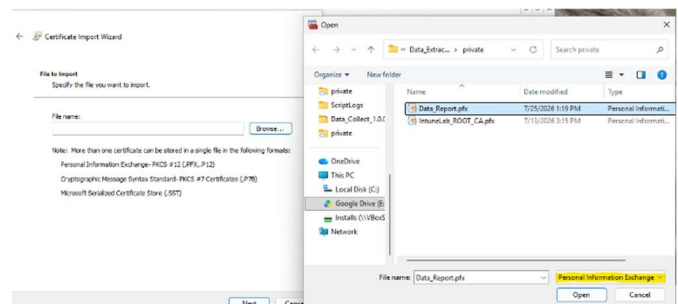
Locate the \Personal\Certificates folder in the left pane. Right-click, chose “All Tasks” and “Import...”



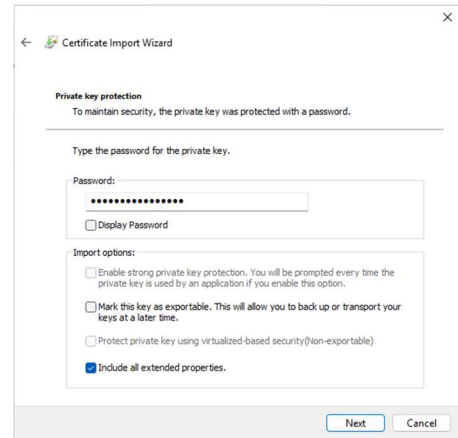
Click “Next”.



Click “Browse”, change the file type to “Personal Information Exchange”, select your .PFX file, click “open” and then click “Next”.

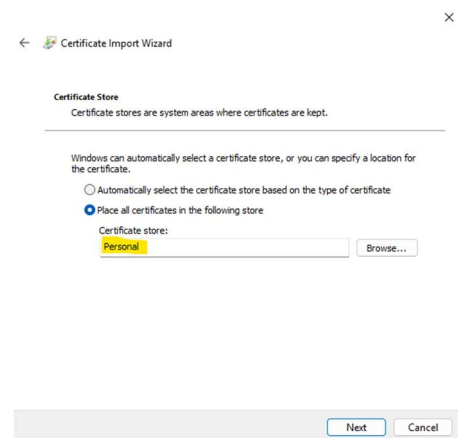


Enter the password and click “Next.”



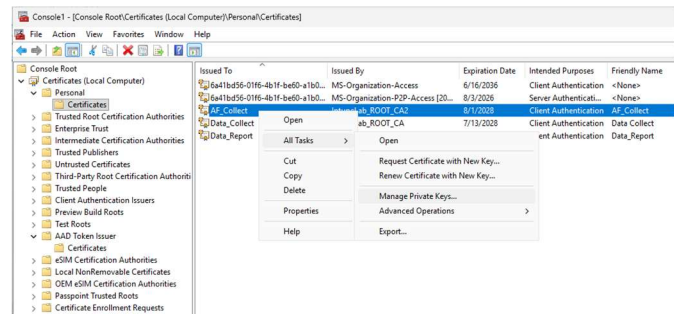
The screenshot shows the 'Certificate Import Wizard' window, specifically the 'Private key protection' step. It explains that the private key is protected with a password. A text box labeled 'Password:' contains a series of dots. Below it is a checkbox for 'Display Password'. The 'Import options' section has three checkboxes: 'Enable strong private key protection...' (unchecked), 'Mark this key as exportable...' (unchecked), and 'Protect private key using virtualized-based security...' (unchecked). The 'Include all extended properties' checkbox is checked. 'Next' and 'Cancel' buttons are at the bottom right.

Make sure you have selected the “Personal” store and click “Next”.



The screenshot shows the 'Certificate Store' step of the wizard. It states that certificate stores are system areas where certificates are kept. It offers two options: 'Automatically select the certificate store based on the type of certificate' (unchecked) and 'Place all certificates in the following store' (checked). Below, the 'Certificate store:' dropdown is set to 'Personal', with a 'Browse...' button next to it. 'Next' and 'Cancel' buttons are at the bottom right.

Right-click on the certificate, choose “All Tasks” and “Manage Private Keys...”.



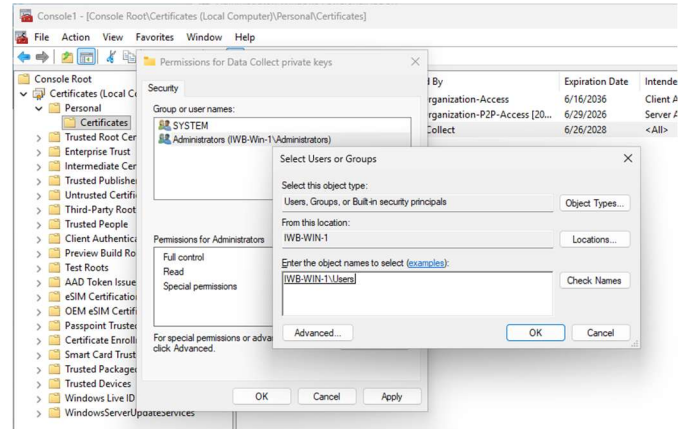
The screenshot shows the 'Certificates (Local Computer)' console window. The left pane shows a tree view with 'Certificates' expanded. The right pane shows a list of certificates. A context menu is open over the 'AF_Collect' certificate, showing options like 'Open', 'All Tasks', 'Cut', 'Copy', 'Delete', 'Properties', and 'Help'. The 'All Tasks' option is highlighted, and a sub-menu is visible showing 'Open', 'Request Certificate with New Key...', 'Renew Certificate with New Key...', 'Manage Private Keys...', 'Advanced Operations', and 'Export...'.

Issued To	Issued By	Expiration Date	Intended Purposes	Friendly Name
ba1bd56-01f6-4b1f-be60-a1b0...	MS-Organization-Access	6/16/2036	Client Authentication	<None>
ba1bd56-01f6-4b1f-be60-a1b0...	MS-Organization-Access [20...	8/3/2036	Server Authentication	<None>
af_collect	ab_ROOT_CA	9/1/2036	Client Authentication	AF_Collect
Data_Collect	ab_ROOT_CA	7/13/2028	Client Authentication	Data_Collect
Data_Report			Client Authentication	Data_Report

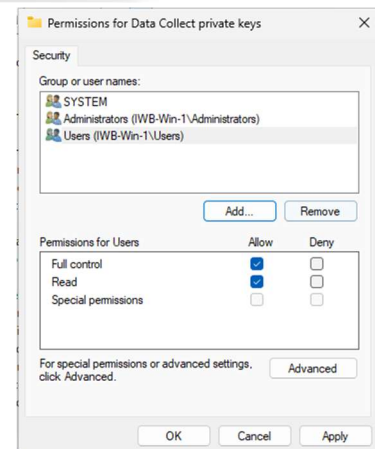
A permissions dialog will pop up. Click “Add”.

Enter “Users” in the bottom box, click “Check Names” to ensure the group or account can be resolved.

Click “OK”.



Verify the group or account has at least “Read” and “Full Control” access. You need at least “Read”, but “Full Control” will be needed to delete the certificate later.



MacOS Certificates

The certificate installation on MacOS is easier to do using Terminal and a few commands. Copy the PFX file for the leaf certificate into your documents folder.

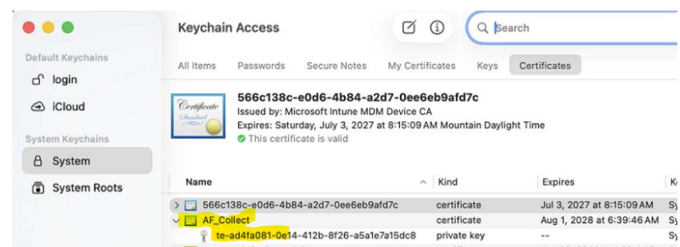
Start the Terminal app and navigate to your documents folder.

Run the following commands:

```
sudo security add-trusted-cert -d -r trustRoot -k/Library/Keychains/System.keychain <Root CA Cert .cer file name>
```

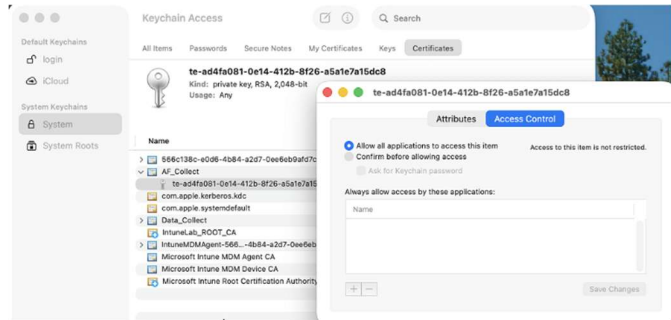
```
sudo security import <Leaf Cert .pfx file name> -k
```

```
/Library/Keychains/System.keychain -P  
"<PFX file password>" -A
```



Using the Keychain Access app, you can now confirm:

- The Leaf certificate is installed with its private key visible.
- “Allow all applications to access this item” is set on the private key.



You can verify the certificate is properly installed if it is listed in the output from “security find-identity -v -p ssl-client”.

Linux Certificates

<This section Pending>

Alter and test the scripts

This is the stage where you modify the Inventory for your own purposes.

Alter the Windows Inventory Script and test

We are going to use a Windows PowerShell script for testing (Windows_Custom_Inventory.ps1). You will need to run the script using administrative access.

Enter “Powershell_ise” in the Windows Search box. Locate the icon for “Windows PowerShell ISE 5.1”, right-click on it and choose “Run as Administrator”.

You will likely need to run “Set-ExecutionPolicy -Scope LocalMachine -ExecutionPolicy Unrestricted” in the console.

Our first goal is to make sure the Inventory script produces a valid JSON file, and the certificate we need can be accessed.

The first few lines of the script are configuration variables.

Populate:

- \$CertName
- \$LogFile

```
1  #  
2  # BEGIN CONFIGURATION  
3  
4  # Modules  
5  Add-Type -AssemblyName System.Web  
6  
7  # Certificate (Required for mTLS)  
8  $CertName = "AF_Collect"  
9  # URL Data (for JSON submission)  
10 $URL = "<URL>"  
11 # Script logging  
12 $LogFile = "C:\ProgramData\ScriptLogs\Inventory.log"  
13 #  
14 # If a log file has been specified and it already exists, delete it.  
15 if (-not [string]::IsNullOrEmpty($LogFile) -and (Test-Path -Litera  
16     try {
```

The next block of code are the functions that provide functionality to find the correct certificate and update the log.

```

157 #
158 # BEGIN SCRIPT
159 #
160 #
161 # Get the first valid certificate that matches $CertName
162 $Cert = Get-ValidCertificate -CertName $CertName
163 # If one was found
164 if ($Cert) {
165     # Get the thumbprint
166     $Thumbprint = (Get-Childitem -Path "Cert:\LocalMachine\My" | where-object {$_.Subject -Match $CertName}).Thumbprint
167     #
168     # BEGIN INVENTORY

```

```
167 #
168 # BEGIN INVENTORY
169 #
170 #
171 #
172 # Get the ManagedDeviceID and Name from Intune enrollment
173 try {
174     # If the device is enrolled
175     if (@(Get-Childitem HKLM:\SOFTWARE\Microsoft\Enrollments) -Recurse |
176         # Get the source for the managed device information
```

```

357 #
422 # Compile all the settings in
423 $Inventory = New-Object System
424 $Inventory Add-Member -Memb
425 $Inventory Add-Member -Memb
426 $Inventory Add-Member -Memb
427 $Inventory Add-Member -Memb
428 $Inventory Add-Member -Memb
429 $Inventory Add-Member -Memb
430 $Inventory Add-Member -Memb
431 $Inventory Add-Member -MemberType NoteProperty -Name "DefaultPage" -Value $DefaultPage -Force
432 $Inventory Add-Member -MemberType NoteProperty -Name "DefaultEngine" -Value $DefaultEngine -Force
433 $Inventory Add-Member -MemberType NoteProperty -Name "BitLockerState" -Value $BitLockerState -Force
434 $Inventory Add-Member -MemberType NoteProperty -Name "BitLockerStart" -Value $BitLockerStart -Force
435 $Inventory Add-Member -MemberType NoteProperty -Name "BitEncrypted" -Value $BitEncrypted -Force
436 $Inventory Add-Member -MemberType NoteProperty -Name "BitEncryption" -Value $BitEncryption -Force
437 $Inventory Add-Member -MemberType NoteProperty -Name "BitProtected" -Value $BitProtected -Force
438 $Inventory Add-Member -MemberType NoteProperty -Name "BitProtector" -Value $BitProtector -Force
439 }
440 foreach ($Sticky in ([NetHash.Keys | Sort-Object]) {
441     $Name = "$MAC + $Sticky.ToString()"
442     $Value = $NetHash[$Sticky]
443     $Inventory | Add-Member -MemberType NoteProperty -Name $Name -Value $Value -Force
444 }
445 }
446 }
447 # Convert to JSON for sending the data to Log Analytics Workspace
448 $Body = $Inventory | ConvertTo-Json
449 # END INVENTORY
450 ""
```

```
PS C:\WINDOWS\system32\cmd [C:\ProgramData\IntuneLab\Projects\Custom_Reporting_AF\Project1]
"ManagedDeviceName": "lanlaxer_windows_6/19/2026_313 1M",
"ManagedDeviceID": "fed2368-73bf-4d6f-9469-09ad63c1c3e",
"DefenderState": "Running",
"DefenderStart": "Automatic",
"DefenderPage": 0,
"DefenisiPage": 0,
"DefenisiPage": 0,
"DefEngine": "1.1.26660.3008",
"BitLockerState": "Zipped",
"BitLockerStart": "Manual",
"BitEncrypted": "Full[Decrypted]",
"BitEncryption": "C:\How",
"BitProtected": "C:\Protected",
"BitProtection": "C:\Unknown",
"XProtect_Version": "C:\Unknown",
"XProtect_Meta": "C:\Unknown",
"XProtect_Launch": "C:\Unknown",
"XProtect_Background": "C:\Unknown",
"FileVault_Status": "C:\Unknown",
"FileVault_UserToken": "C:\Unknown",
"FileVault_BootToken": "C:\Unknown",
"BuildDate": "2026-06-16T07:25:30",
"LastUpdateTime": "2026-06-03T07:30:31",
"WACD": "Intel(R) P60/1600 MT Desktop Adapter [08:00:27:CA:49:3C]:",
"WAC2": "C:\",
"WAC3": "C:\",
"WAC4": "C:\",
"WAC5": "C:\",
}
```

If you did everything correctly, you should see the JSON contents.

When you have proven the script works, you can comment out the debugging code.

Alter the MacOS Inventory Script and test

```

5 #-----
6 # CONFIGURATION
7 #-----
8 # A substring we use to find the private key
9 IDENTITY="AF_Collect"
10 # URL (for JSON submission)
11 URL="<URL>"
12 # LOG FILE
13 logFile="/Library/Microsoft/CustomInventory.log"
14 #-----
15 # begin initialization

```

- IDENTITY
- LogFile

Customize the Inventory block to collect the data you want to report via JSON upload. If you are reporting for both Windows and MacIntosh, remember that your JSONs must have identical structures and plan accordingly.

Use the code example to create the structure we are going to export to JSON.

```
185 )
186 #####
187 # Begin Inventory script
188 #####
189 #-----
190 # Detect Intune Device Name
191 #-----
192 MANAGEDDEVICENAME=$(Hostname)
193 #
194 #-----
```

```
366 #-----
367 # Build json
368 #-----
369 jsonData=$(jq -n \
370   --arg ManagedDeviceName "$MANAGEDDEVICENAME" \
371   --arg ManagedDeviceID "$MANAGEDDEVICEID" \
372   --arg DefenderState "" \
373   --arg DefenderStart "" \
374   --arg DefSpySigAge "" \
```

After the inventory data has been converted to JSON, we have a block where breakpoints can be set to allow debugging.

Uncomment this code.

```
307 #
308 #
309 # Debugging
310 echo "$jsonData"
311 exit 0
312 #
313 #-----
```

Run the Inventory script using “sudo bash MacOS_Inventory.sh”.

If you did everything correctly, you would see the JSON contents followed by the access token you received.

If not, you need to fix something either with your certificate or with your inventory code.

```
ianbaxter@Ians-MacBook-Pro documents % sudo bash MacOS_Custom_Inventory.sh
Password:
{
  "ManagedDeviceName": "Ians-MacBook-Pro.local",
  "ManagedDeviceID": "566c138c-e0d6-4b04-a2d7-8ee6b9afd7c",
  "DefenderState": "",
  "DefenderStart": "",
  "DefSpySigAge": "",
  "DefPNSigAge": "",
  "DefAVSigAge": "",
  "DefAMEngine": "",
  "BitLockerState": "",
  "BitLockerStart": "",
  "BitEncryption": "",
  "BitProtected": "",
  "BitProtector": "",
  "XProtect_Version": "5353",
  "XProtect_Meta": "5353",
  "XProtect_Launch": "enabled",
  "XProtect_Background": "enabled",
  "FileVault_Status": "Macintosh HD - Data: Not Encrypted",
  "FileVault_UserToken": "No console user",
  "FileVault_BootToken": "Encrypted",
  "BuildDate": "07/02/2026, 12:56:49 PM",
  "LastBootTime": "08/02/2026, 07:19:58 AM",
  "MAC0": "Thunderbolt Bridge 12:50:5b:5b:e6:c1",
  "MAC1": "Wi-Fi 00:0c:aa:07:70:e4",
  "MAC2": "Thunderbolt 1 12:50:5b:5b:e6:c1",
  "MAC3": "Thunderbolt 2 12:50:5b:5b:e6:c0",
  "MAC4": "Thunderbolt 3 12:50:5b:5b:e6:c5"
}
ianbaxter@Ians-MacBook-Pro documents %
```

Comment out the breakpoint when you have it working.

Alter the Linux Inventory Script and test

<This section Pending>

Set up Log Analytics

Schema (JSON)

In the older version of Log Analytics Data Ingestion API, your JSON could be fluid. MacOS and Windows devices typically have different disk encryption and anti-virus products and each platform could use differently formatted JSON files. The table structure would change in response to these differences. This is no longer the case.

The new API submits the JSON via a Data Collection EndPoint (DCE) which passes it to a transform process, the Data Collection Rule (DCR). Data is then filtered to ensure that anything passed to Log Analytics matches an existing table structure and has been submitted using the Azure Function's Managed Identity credentials. This means that MacOS and Windows clients must create identically formatted JSON output and submit it securely using mTLS to allow us to detect and prevent spoofing of compliance data.

This example focuses on disk encryption, anti-virus and physical network adapters since Intune only reports on wi-fi adapter details.

The Intune hardware inventory collects once in 7 days. You can use the Extended Inventory to reduce that to every one day for details it can collect. Custom Compliance scripts can be set to run every hour, proving a much more real-time data source for live dashboards.

If you have deployed Microsoft Defender to MacOS devices and don't have access to the Microsoft Defender for Endpoint Graph API's, you may have noticed the Intune Graph API for Defender only lists data for Windows devices. If you need to collect Defender data for Windows and MacOS, this may be your only option.

The DCE expects a JSON array of these objects, even if only one JSON item is in the array. The Table and DCR creation processes only use one JSON item. Windows devices can report blanks for MacOS-specific metrics and vice-versa. Make sure typical values are populated for all values in this JSON template, both Windows and MacOS values need to be combined.

Here is our template:

```
{
  "ManagedDeviceName": "IanBaxter_Windows_6/19/2026_3:13 AM",
  "ManagedDeviceID": "<INTUNE DEVICE ID>",
  "DefenderState": "Running",
  "DefenderStart": "Automatic",
  "DefSpySigAge": 1,
  "DefNisSigAge": 1,
  "DefAVSigAge": 1,
  "DefAMEngine": "1.1.26060.3008",
  "BitlockerState": "Running",
  "BitlockerStart": "Manual",
  "BitEncrypted": "C:FullyDecrypted",
```

```

"BitEncryption": "C:None",
"BitProtected": "C:Unprotected",
"BitProtector": "C:Unknown",
"XProtect_Version": "5346",
"XProtect_Meta": "5346",
"XProtect_Launch": "enabled",
"XProtect_Background": "enabled",
"FileVault_Status": "Macintosh HD - Data:Encrypted",
"FileVault_UserToken": "enabled",
"FileVault_BootToken": "enabled",
"BuildDate": "2026-06-16T17:25:30",
"LastBootTime": "2026-07-08T13:50:03",
"MAC0": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC1": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC2": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC3": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC4": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC5": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C"
}

```

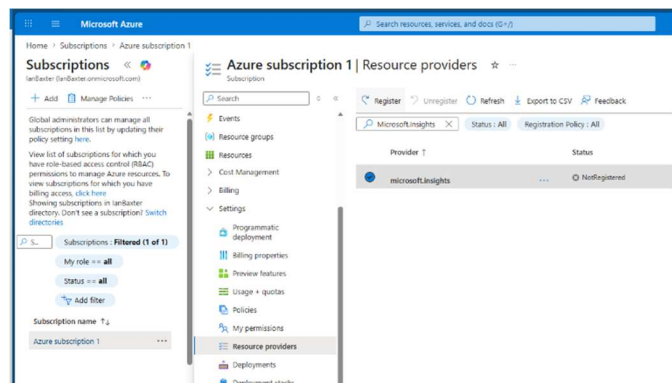
Register Microsoft Insights resource provider

In the Azure Portal, search for
“Subscription” and load the applet.

Select the correct subscription. Click on
“Resource Providers”, search for
“Microsoft.Insights”.

Select it and click the “Register” option.

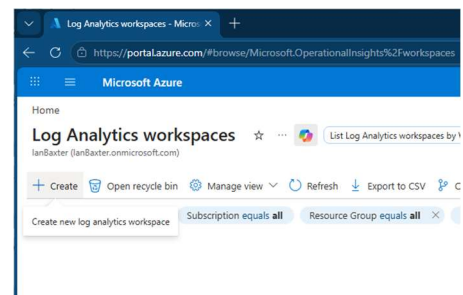
Wait for the notification that the
registration has completed.



Create the Log Analytics Workspace

Launch the Azure Portal, search for Log Analytics
workspaces.

Click on “Create”.



You may need to create a Resource Group. If so, click the “Create New” option.

Populate the Name and Region, click “Review + Create”.

When the validation passes, click “Create”.

Click the “Go to Resource” option.

The screenshot shows the 'Project details' section of the Azure portal. It includes a 'Subscription' dropdown set to 'Azure subscription 1' and a 'Resource group' dropdown. A modal dialog is open, explaining that a resource group is a container for related resources and prompting the user to enter a name. The name 'Intune-Reporting' is entered in the 'Name' field. Below the modal, the 'Instance details' section shows 'Name' and 'Region' fields. At the bottom, there are buttons for 'Review + Create', '< Previous', and 'Next: Tags >'. The modal has 'OK' and 'Cancel' buttons.

Create a DCE (Data Collection Endpoint)

Open the Azure Portal, search for “Data Collection Endpoints” and launch the applet.

Click the “Create” option.

The screenshot shows the 'Data collection endpoints' page in the Azure portal. It includes a 'Create' button, a 'Manage view' dropdown, and a 'Refresh' button. There are also links for 'Export to CSV', 'Open query', and a 'Identify non-compliant Data' button. Below these, there are filters for 'Subscription equals all' and 'Resource Group equals all'. A 'Create new data collection endpoint' button is visible.

Populate the “Endpoint Name”, select the appropriate Subscription, Resource Group and Region.

The Region should be the same as your Tenant and must match the values for the DCR and Log Analytics workspace.

Click “Review and Create”.

The screenshot shows the 'Create data collection endpoint' form in the Azure portal. It includes a 'Basics' tab and a 'Review + create' button. The form has fields for 'Endpoint Name' (set to 'Custom-Compliance'), 'Subscription' (set to 'Azure subscription 1'), 'Resource Group' (set to 'Intune-Reporting'), and 'Region' (set to 'Canada Central'). There are also links for 'Learn more' and 'Create new'. At the bottom, there are buttons for 'Review + create', '< Previous', and 'Next: Tags >'. The form also includes a 'Tags' tab and a 'Review + create' button.

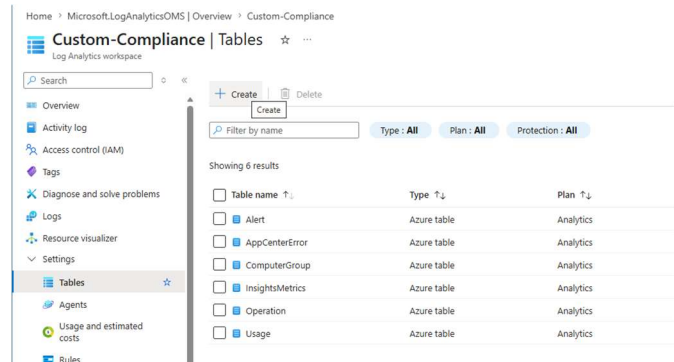
Create the Log Analytics Table and DCR

Let me caution you here. You need to do this once; do it right and fully understand what you are doing at this step. It seems simple from all the Internet documentation, but it really is not and is poorly documented.

If you make a mistake, you cannot delete and re-use the old table and DCR names. The data that underlies them can persist for up to 90 days. The data for the table or DCR does not get deleted and will re-populate in any new one you recreate with the same name. The only other solution is to create one with a different name.

With the log analytics workspace open, expand the “Settings” and select “Tables”.

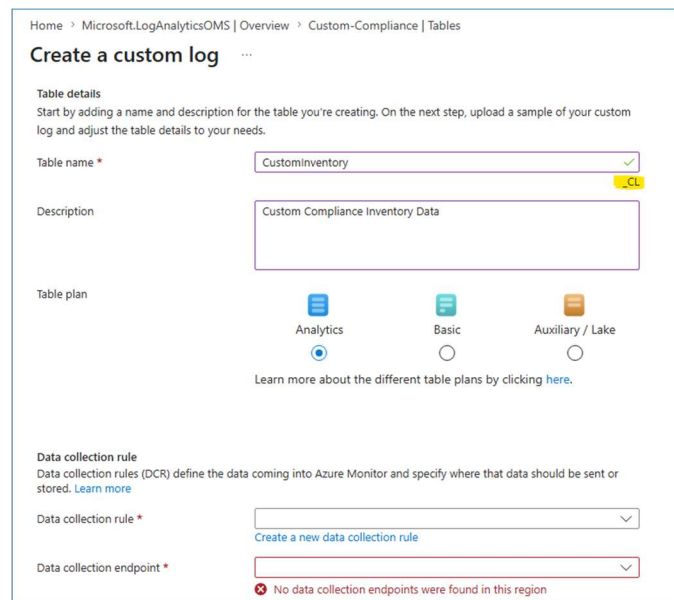
Click the “Create” option.



The table name must be the same as the table you will configure in the script with a few differences. The script uses a stream name which is “Custom-<TableName>_CL”. The “_CL” gets appended to the table name for “Custom Log” in Log Analytics, and the “Custom-” gets added by the DCR to form the data stream name.

Enter a description.

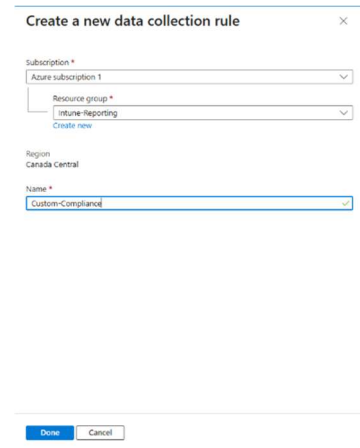
You should not have a DCR yet, so click the “Create a new data collection rule” option.



Choose your subscription and resource group, give the DCR a name.

Click “Done”.

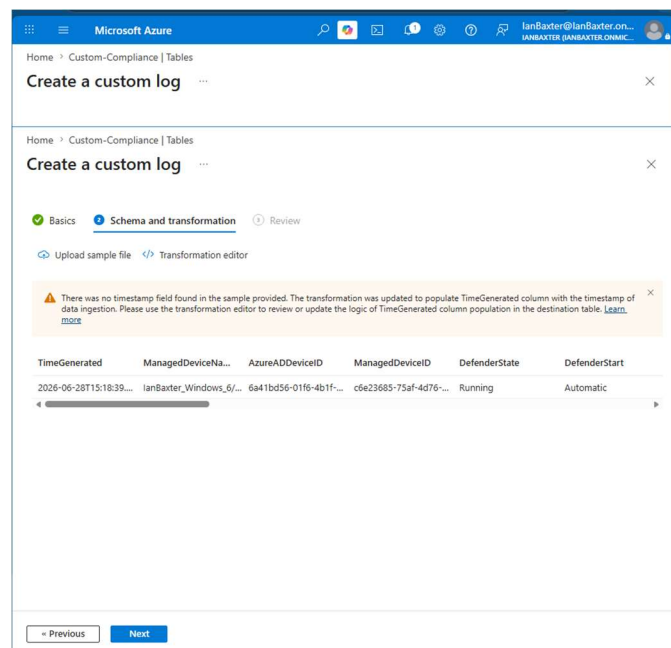
Select the Data Collection Endpoint you created earlier, click “Next”.



The screenshot shows the 'Create a new data collection rule' form. It includes fields for 'Subscription' (set to 'Azure subscription 1'), 'Resource group' (set to 'Azure-Reporting'), and 'Name' (set to 'Custom-Compliance'). The 'Region' is set to 'Canada Central'. At the bottom, there are 'Done' and 'Cancel' buttons.

Upload the Json data you saved from the script test.

You should see the warning that a transform has been created for TimeGenerated. If your JSON doesn't contain the mandatory TimeGenerated data, a transform will be automatically created for you.



The screenshot shows the 'Create a custom log' form in the 'Schema and transformation' tab. A warning message states: 'There was no timestamp field found in the sample provided. The transformation was updated to populate TimeGenerated column with the timestamp of data ingestion. Please use the transformation editor to review or update the logic of TimeGenerated column population in the destination table. [Learn more](#)'. Below the warning is a table with the following data:

TimeGenerated	ManagedDeviceNa...	AzureADDeviceID	ManagedDeviceID	DefenderState	DefenderStart
2026-06-28T15:18:39...	IanBaxter_Windows_6/...	6a41bd56-01f6-4b1f-...	c6e23685-75af-4d76-...	Running	Automatic

At the bottom, there are 'Previous' and 'Next' buttons.

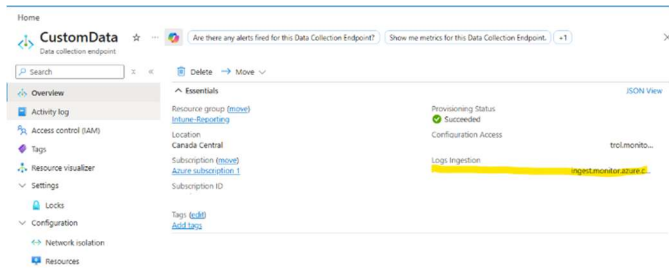
You will also see all the names of values in the JSON you uploaded.

Click “Next” and “Continue”.

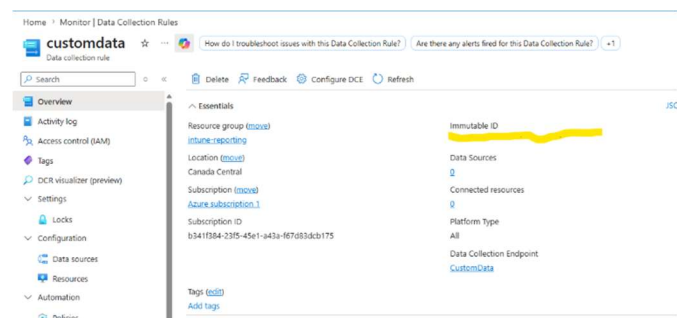
Collect the DCE/DCR details you'll need in code

Record the Logs Ingestion URL from the DCE Overview somewhere so you can use it to configure all the scripts.

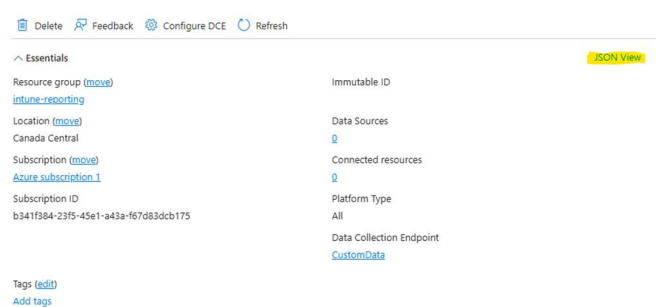
Make a note of the location and Resource Group as well.



Record the DCR Immutable ID from the DCR Overview.



In the DCR Overview, click the "JSON View" link.



Look for this section of the code and record the stream name.

```
9 |         "dataCollectionEndpointId": "/subscriptions/b341f384-23f5-45e1-a43a-f67d8  
10 |         "streamDeclarations": {  
11 |             "Custom-CustomData-CL": {  
12 |                 "columns": [  
13 |                     {  
14 |                         "name": "TimeGenerated",  
15 |                         "type": "datetime"  
16 |                     },  
17 |                 ],  
18 |             }  
19 |         }
```

Set up the Azure Function Script

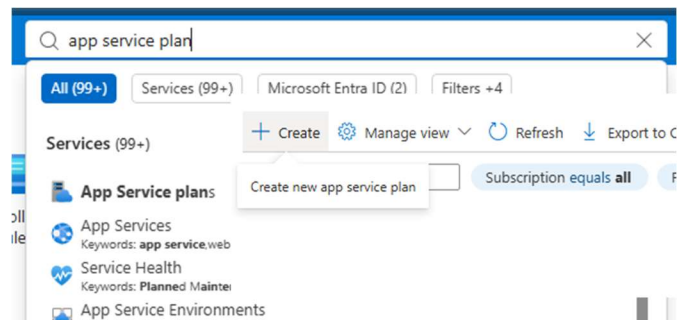
The first step is to configure the Azure Function and its managed identity. In order to use an Azure function, you need a VM to run it on, and this requires an App Service Plan. In a small lab like mine, it is a significant cost. In a large infrastructure, one can expect that this cost can be shared amongst many projects which requires more VM's and more infrastructure, but the costs

can be shared due to scale. Unfortunately, the F1 and D1 plans do not support mTLS. You can find the features and costs for each plan type here: [App Service Pricing | Microsoft Azure](#)

Create the App Service Plan (required for mTLS)

For our short-term project, the minimum plan that supports mTLS is the B1 plan. This is what we are going to use.

Search for “App Service Plans” and start the app.



Click the “+ Create” option.

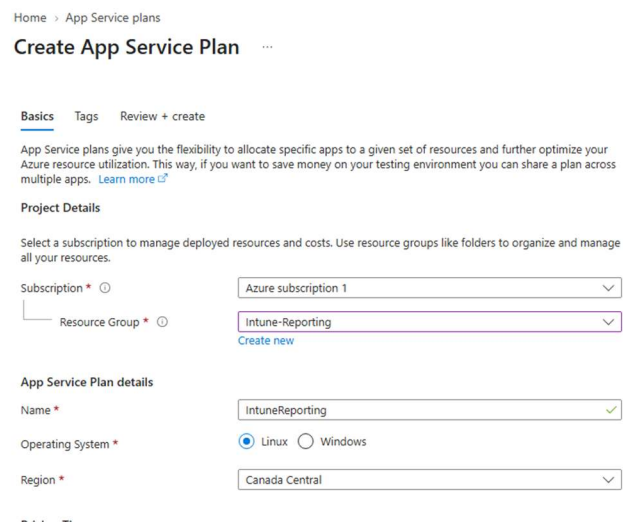
Make sure you have the correct Subscription selected.

Select or create your resource group.

Give the plan a name.

In this case, since we will be using Python, we will select “Linux” for the operating system. For Powershell or C#, select “Windows”.

Make sure the region is the same as your DCE/DCR.



Scroll down and select the Pricing Plan.

Click “Review and Create.”

Review the plan details to make sure they are correct and click “Create.”

Wait for the process to complete.

Pricing Tier

App Service plan pricing tier determines the location, features, cost and compute resources associated with your app. [Learn more](#)

Pricing plan: Basic B1 (1.75 GB memory, 1 vCPU) [Explore pricing plans](#)

Zone redundancy

An App Service plan can be deployed as a zone redundant service in the regions that support it. Your initial instance count will be set based on your zone redundancy configuration. To ensure you'll be able to enable zone redundancy at any point in the lifecycle of your app, enable zone redundancy now. You can decrease the instance count after the App Service plan is created. [Learn more](#)

☐ Zone redundancy is not available on the selected pricing tier.

Zone redundancy

☐ Enabled: Your App Service plan and the apps in it will be zone redundant. The minimum App Service plan instance count will be two.

☒ Disabled: Your App Service plan and the apps in it will not be zone redundant. The minimum App Service plan instance count will be one.

[Review + create](#) [< Previous](#) [Next: Tags >](#)

Delete Cancel Redeploy Download Refresh

✓ Your deployment is complete

Deployment name: Microsoft.Web-ASP-Portal-524a0a88-8653 Start time: 8/8/2026, 8:32:24 AM
Subscription: Azure subscription 1 Correlation ID: c368468f-4eca-4c78-a432-acc0ea1c17f3
Resource group: Intune-Reporting

Deployment details

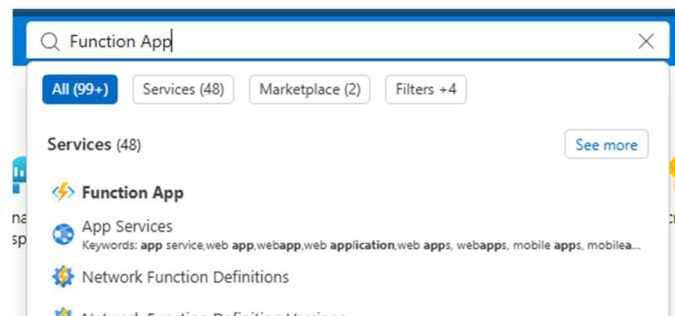
Resource	Type	Status	Operation details
IntuneReporting	Microsoft.Web/serverfarms	OK	Operation details

Next steps

[Go to resource](#)

Create the Function App

In the Azure portal, search for “Function App” and load the app.



Click the “+ Create” option.

Function App ★ ... [Identify web apps with failures](#)

lanBaxter (lanBaxter.onmicrosoft.com)

[+ Create](#) [Manage view](#) [Refresh](#) [Export to CSV](#) [Op](#)

Create new app service Id... Subscription equals all Resource Gro

We will use the “Flex Consumption” hosting option.

Home > Function App

Create Function App

Select a hosting option

These options determine how your app scales, resources available per instance, and pricing. [Learn more about Functions hosting options](#)

Hosting plans	Flex Consumption	Functions Premium	App Service Plan
	Get high scalability with compute choices, virtual networking, and pay-as-you-go billing.	Deploy multiple function apps on the same plan with event-driven scaling.	Run web apps on the same plan with the same choices as the plan.
Scale to zero	✓	-	-
Scale behavior	Fast event-driven	Event-driven	Metric
Virtual networking	✓	✓	✓
Dedicated compute and prevent cold start	Optional with Always Ready	Minimum of 1 instance required	Minimum
Max scale out (instances)	1000	100	30

Select

Select the Resource Group.

Give the Function App a name. Note the URL that is formed.

Select “Python” for the runtime stack.

Select whichever version of Python you will use. Only 3.11 and 3.10 are supported.

Scroll down to select the Zone Redundancy option. Since this is a lab, we disable this.

You need a Storage Account, click “Next.”

Create Function App (Flex Consumption)

Basics Storage Azure OpenAI Networking Monitoring Durable Functions Deployment Authentication

Create a function app, which lets you group functions as a logical unit for easier management, deployment and sharing of resources. Functions lets you execute your code in a serverless environment without having to first create a VM or publish a web application.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Azure subscription 1

Resource Group * Intune-Reporting
[Create new](#)

Instance Details

Function App name * Intune-Reporting
h9erhpgbhnfybqf0.canadacentral-01.azurewebsites.net

Region * Canada Central

Runtime stack * Python

Version * 3.14

Instance size * 2048 MB

Zone redundancy

Instances of your app are distributed across availability zones for increased reliability. [More about zone redundancy.](#)

Zone redundancy

☐ Enabled: Your Flex Consumption app will be zone redundant. This changes your app's required instance count per function or function group.

☒ Disabled: Your Flex Consumption app will not be zone redundant.

[Review + create](#) < Previous Next : Storage >

We are going to accept the default name and setting to configure Blob diagnostic settings later.

Click “Next.”

We don’t need Azure AI. Click “Next.”

Because we are in a lab and this function app is not going to last very long, we can accept the default network settings.

Home > Function App > Create Function App

Create Function App (Flex Consumption) ...

Basics **Storage** Azure OpenAI Networking Monitoring Durable Functions Deployment Authentic

Storage

Select a storage account or create a new one. Accounts must support blobs, queue, and Table storage. [Learn more](#)

Storage account * [Create new](#)

i You can configure networking on new storage accounts when you enable virtual network integration and configure outbound access networking on your function app.

Diagnostic Settings

Configure diagnostic settings to enable data monitoring for your storage account. [Learn more](#)

Blob service diagnostic settings

☒ **Configure later (Recommended for custom controls)** Select this option for control over log destinations, retention policies, and specific logs and metrics.

☐ **Configure now (Recommended for basic controls)** Select to configure Azure Log Analytics with storage, write logs, and transaction metrics for the blob service. You can still modify these settings later.

[Review + create](#) [< Previous](#) [Next : Azure OpenAI >](#)

Home > Function App > Create Function App

Create Function App (Flex Consumption) ...

Basics Storage **Azure OpenAI** Networking Monitoring Durable Functions

Add Azure OpenAI resource and vector database (optional) to create intelligent apps. [Learn More](#)

Enable Azure OpenAI * ☐

Home > Function App > Create Function App

Create Function App (Flex Consumption) ...

Basics Storage Azure OpenAI **Networking** Monitoring Durable Functions Deployment

Function Apps can be provisioned with the inbound address being public to the internet or isolated to an Azure virtual network. Function Apps can also be provisioned with outbound traffic able to reach endpoints in a virtual network, be governed by network security groups or affected by virtual network routes. By default, your app is open to the internet and cannot reach into a virtual network. These aspects can also be changed after the app is provisioned. [Learn more](#)

i You can configure networking on new services when you enable virtual network integration and configure outbound access networking on your function app.

Enable public access * ☒ On ☐ Off

Enable virtual network integration * ☐ On ☒ Off

We are going to disable Application Insights for now.

Click “Next.”

For the lab, we don’t need durable functions.

Click “Next.”

Click “Next” to skip the Deployment options.

Select “Managed Identity” for both authentication options. We are going to accept the default Managed Identity name instead of specifying one.

Click “Next” to skip past the Tags.

Review the settings and click “Create.”

Once created, click on “Go to Resource.”

Home > Function App > Create Function App

Create Function App (Flex Consumption) ...

BasicsStorageAzure OpenAINetworkingMonitoringDurable Functions

Application Insights

Azure Monitor application insights is an Application Performance Management (APM) service for dev DevOps professionals. Enable it below to automatically monitor your application. It will detect perform and includes powerful analytics tools to help you diagnose issues and to understand what users actu app. Your bill is based on amount of data used by Application Insights and your data retention setting

App Insights pricing

Enable Application Insights *☒ No ☐ Yes

Home > Function App > Create Function App

Create Function App (Flex Consumption) ...

BasicsStorageAzure OpenAINetworkingMonitoringDurable FunctionsDeploymentAuther

Create a Durable Task Scheduler resource ☐

If disabled, Durable Functions uses the Azure Function app's default storage account instead. This supports only the most basic runtime features and has limited throughput. [Compare storage provider options.](#)

Home > Function App > Create Function App

Create Function App (Flex Consumption) ...

BasicsStorageAzure OpenAINetworkingMonitoringDurable FunctionsDeploymentAuthenti

Resource authentication

For best security practice, use managed identity authentication when available (some resources may only use secrets).

Resource	Name	Authentication type	Minimum roles required
Host storage (AzureWebJobsStorage)	intunereportingbcdca	Managed identity	Storage Blob Data Owner
Deployment storage	app-package-intune-reporting-ec2e1fd	Managed identity	Storage Blob Data Contributor

Managed identity

Assign a managed identity for your app. You can adjust these settings later.

Managed identity *

[Create new managed identity](#)

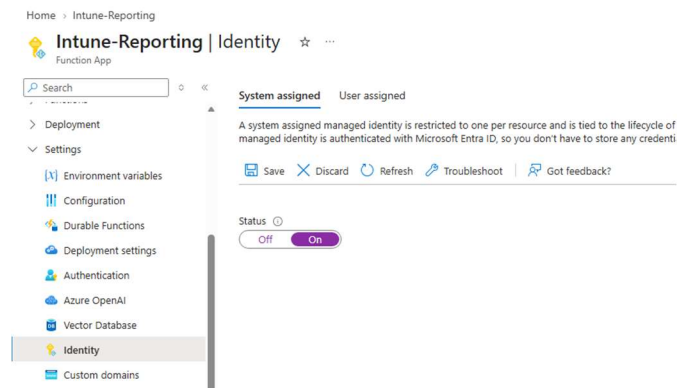
☒ Suggested roles will be automatically assigned for new managed identities.

Review + create< PreviousNext : Tags >

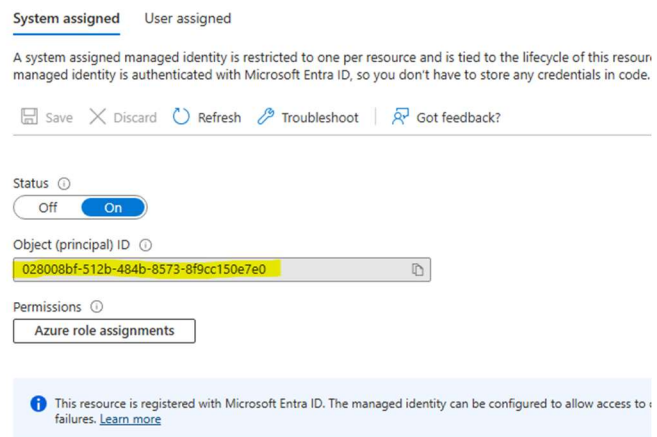
Enable System Assigned Managed Identity

With the Function App open, expand the “Settings” and click “Identity”. Turn on the “System assigned” option.

Click “Save.”



Once the Managed Identity is created, you will be shown the Service Principal ID. Record this for later use. This identity will later be granted RBAC on your DCR/DCE.

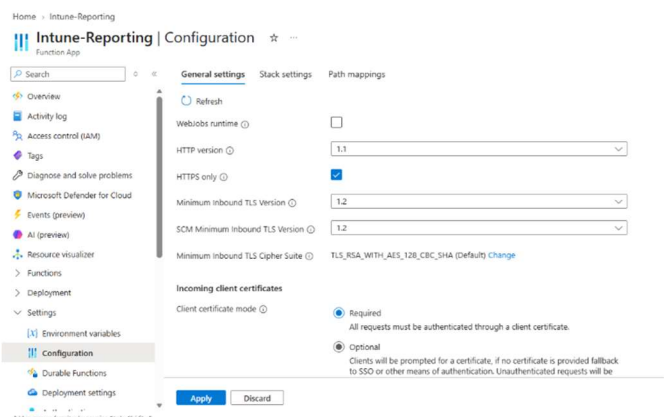


Enable mTLS (Client Certificate Mode) and Disable HTTP

Select “Configuration” in the left pane.

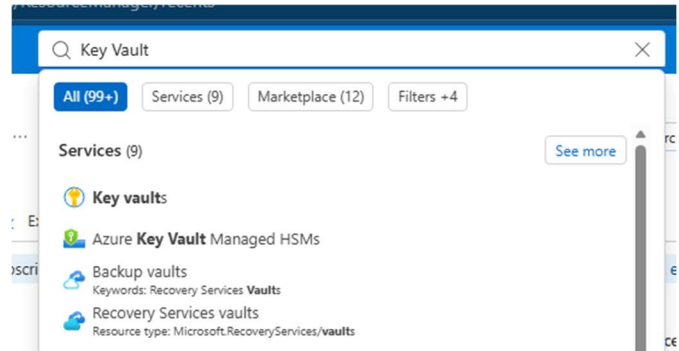
Ensure “HTTPS Only” is selected and “Client certificate mode” is set to “Required.”

Click “Apply.”

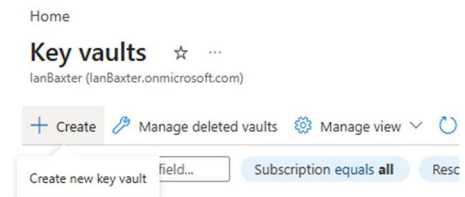


Create an Azure Vault and store the Root CA PEM

In the Azure Portal, search for Key Vaults and load the app.



Click on the “+ Create” option.



Select your Resource Group, give the key vault a unique name and ensure you have it in the same region as your DCE/DCR.

Home > Key vaults

Create a key vault

Project details
Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group * [Create new](#)

Instance details

Key vault name * ✓

Region *

Pricing tier *

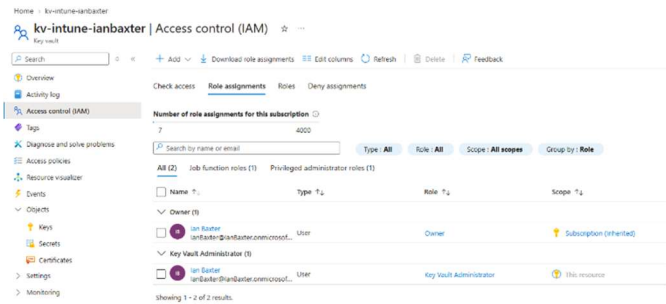
Recovery options
Soft delete protection will automatically be enabled on this key vault. This feature allows you to recover or permanently delete a key vault and secrets for the duration of the retention period. This protection applies to the key vault and the secrets stored within the key vault.

To enforce a mandatory retention period and prevent the permanent deletion of key vaults or secrets prior to the retention period elapsing, you can turn on purge protection. When purge protection is enabled, secrets cannot be purged by users or

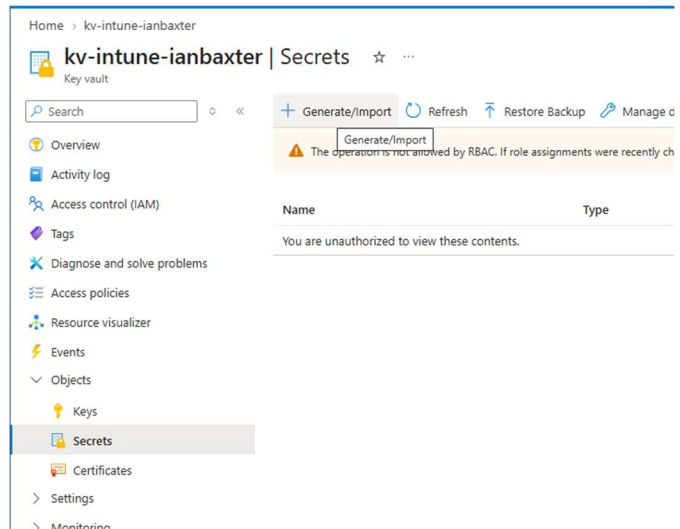
[Previous](#) [Next](#) [Review + create](#)

Click “Next.”

Select “Identity and Access Management” and add your account with the “Key Vault Administrator” role.

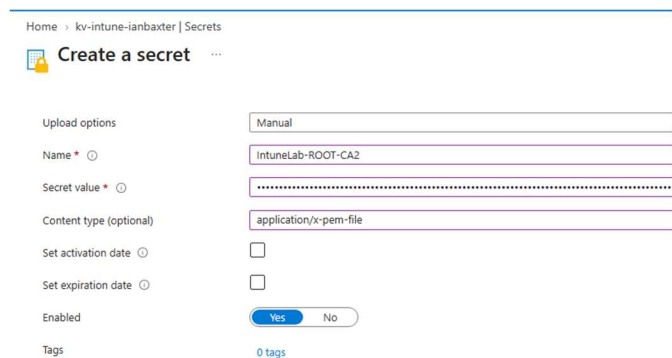


With the Key Vault open, expand “Objects” in the left pane, select “Secrets” and click the “+ Generate/Import” option.



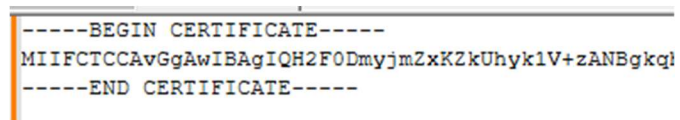
Enter the name for the setting, paste the contents of the PEM file into the “Secret Value” and set the “Content Type” to “application/x-pem-file.”

Note: The PEM contents should all be on one line without carriage returns or line breaks.



Example:

Click “Create.”



Assign RBAC permissions to the Managed Identity

Data Collection Rule

Launch the Azure portal, locate and click on the Data Collection Rule you created.

Select “Access control (IAM)” in the left name and click the “+ Add” option.

Search for and select the “Reader” role.

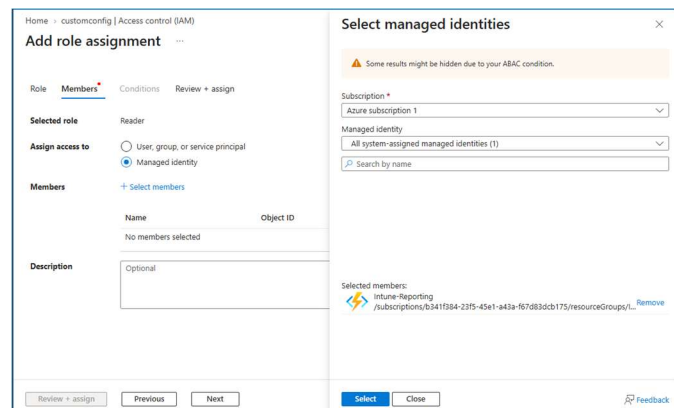
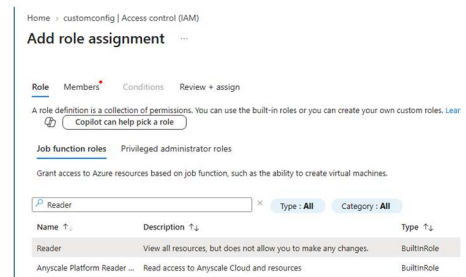
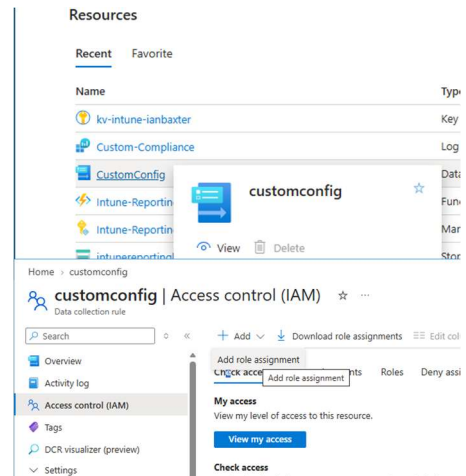
Click “Next”.

Select the “Members” tab, ensure the “Assign access to” option “Managed identity” is selected.

Click the “+ Select members” link.

In the Managed identity” dropdown, select “All system-assigned managed identities”.

Click on the managed identity to add it to the “Selected members” list.



Click the “Select” button.

Review the results.

Click “Review and assign”.

Click “Review and assign” again.

Home > customconfig | Access control (IAM)

Add role assignment ...

Role Members Conditions Review + assign

Selected role Reader

Assign access to ☐ User, group, or service principal ☒ Managed identity

Members + Select members

Name	Object ID	Type
Intune-Reporting	028008bf-512b-484b-8573-8f9cc15...	Function App

Description Optional

If you click on the “Role Assignments” tab, you can verify the role has been added.

Search

+ Add Download role assignments Data columns Refresh Delete Filter

Search by name or email

Type: All Role: All Scope: All

Group by: Role

Role	Role	Scope
Intune-Reporting	028008bf-512b-484b-8573-8f9cc15...	Function App
Monitoring Metrics Publisher		

Repeat this process to add the “Monitoring Metrics Publisher” role.

Log Analytics Workspace

Locate and open your Log analytics workspace in the Azure portal.

Resources

Recent Favorite

Name	Type
Custom-Compliance	Custom-Compliance
ASP-IntuneReporting	
IntuneReporting	

Give your Managed Identity the “Log Analytics Contributor” role.

Home > Custom-Compliance | Access control (IAM)

Add role assignment ...

Role Members Conditions Review + assign

Role Log Analytics Contributor

Scope /subscriptions/b341f384-23f5-45e1-a43a-67d83dc175/resourceGroups/Intune-Reporting/providers/Microsoft.OperationalInsight/Compliance

Members

Name	Object ID	Type
Intune-Reporting	028008bf-512b-484b-8573-8f9cc150e7e0	Function App

Description No description

Key Vault

Locate and open your Key vault in Azure portal.

CustomContig

kv-intune-ianbaxter

Intune-Reporting

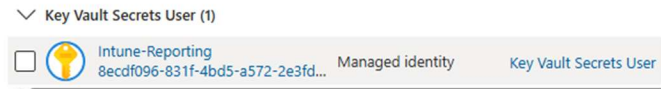
Intune-Reporting-uar

intunereportingbcd

kv-intune-ianbaxter

View

Give your Managed Identity the “Key Vault Secrets User” role.



Azure Function Script (Python)

All of the pre-requisites are in place except for the actual Azure function. We are going to create it, test its basic functionality and then expand it, testing each iteration in order to reduce the number of possible failure points. There are differences between Python, PowerShell and C# azure functions and how they are built and deployed.

The easiest way to create and edit the python Azure Function is by using Visual Studio Code.

Install and Configure VS Code

Install VS Code from this link: [Visual Studio Code - The open source AI code editor | Your home for multi-agent development](#)

Install Azure CLI

You can find the .MSI download like here: <https://aka.ms/installazurecliwindows>

Install the version of Python you will be using

Download the same version of Python you configured the Function App to use from here:

[Download Python | Python.org](#)

Run the installer. Make sure you select “Add python.exe to the PATH.”

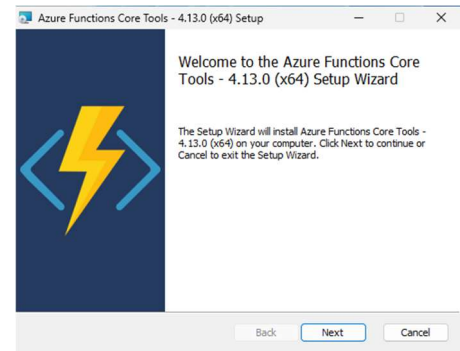
Click “Install Now.”



Install Azure Functions Core Tools (v4.x)

The installation for Azure Functions Core Tools can be found at this link: [Develop Azure Functions Locally by using Core Tools | Microsoft Learn](#)

Download and install using the default settings.

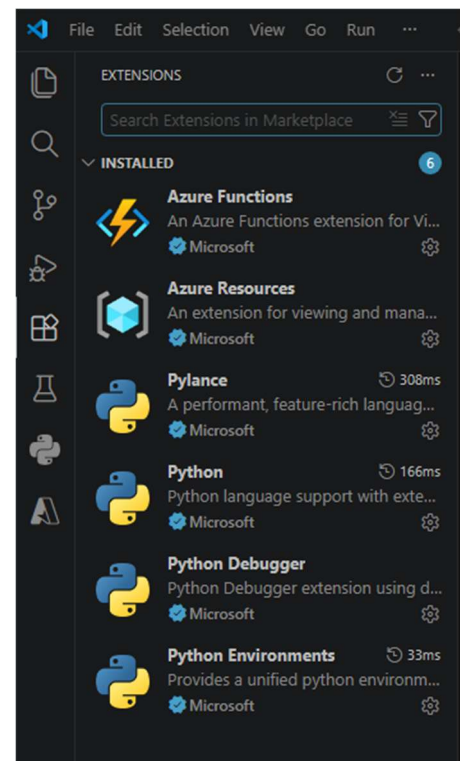


Add VS Code Extensions

Launch VS Code and install the following extensions:

- Azure Functions (by Microsoft)
- Python (by Microsoft)

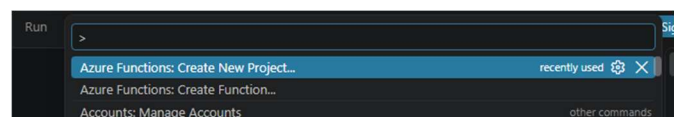
The rest of the listed extensions will be installed automatically.



Create the Azure Function (Data Collect)

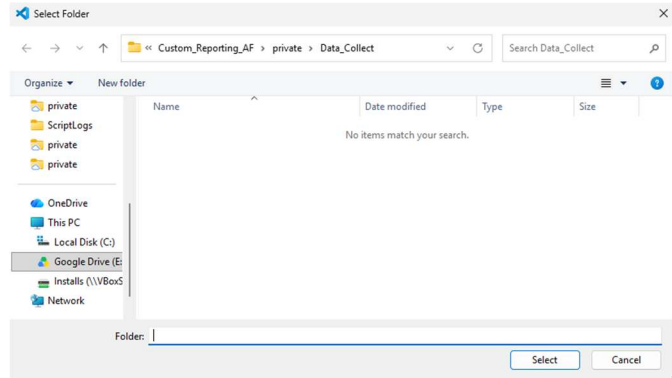
Create an empty folder to contain the project.

Launch Visual Studio Code and hit the "F1" key. From the pulldown menu that appears, select "Azure Functions: Create New Project..."

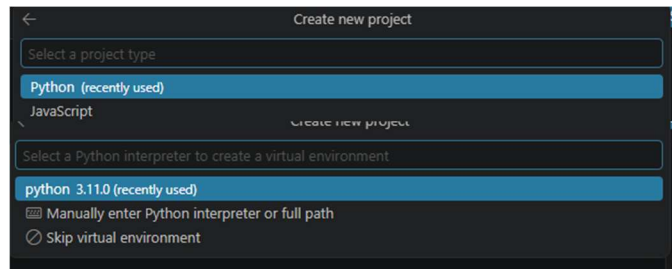


Browse to your project folder.

Click “Select.”

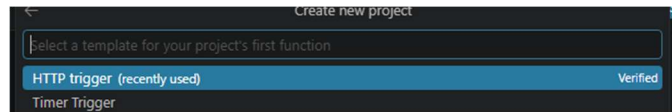


Select “Python.”

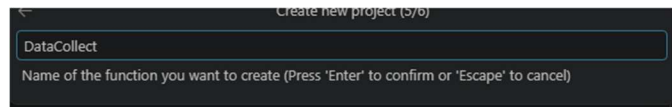


Select “Python 3.11”.

Select “HTTP Trigger.”

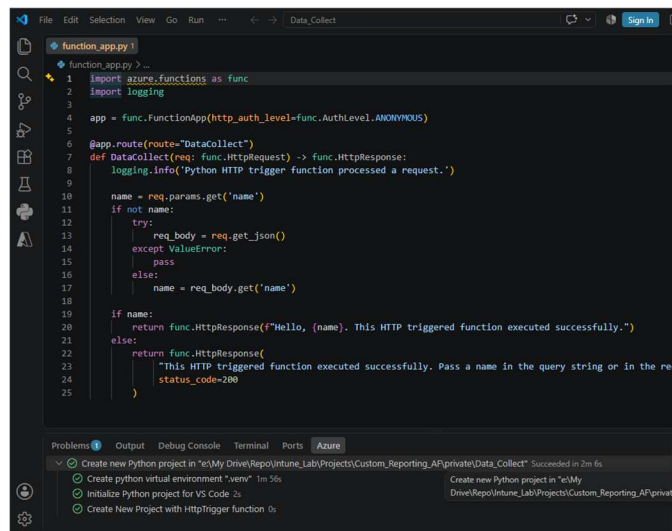


Give the function a name.



Select “Anonymous” for the authorization level controls. This is because we will authenticate using a certificate and mTLS.

The project will then get created with a sample Python script you can use to test locally.



Test Locally with func

Switch to the Terminal tab, wait for the virtual environment to activate and type “func start” and hit enter.

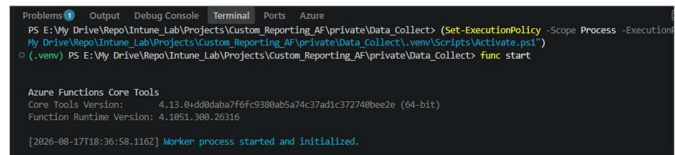
You may have to allow func in a pop-up Windows Firewall prompt.

To avoid having to install Curl, I created a PowerShell script

“Windows_Local_Connect_Tester.ps1” that can be used to verify the script works.

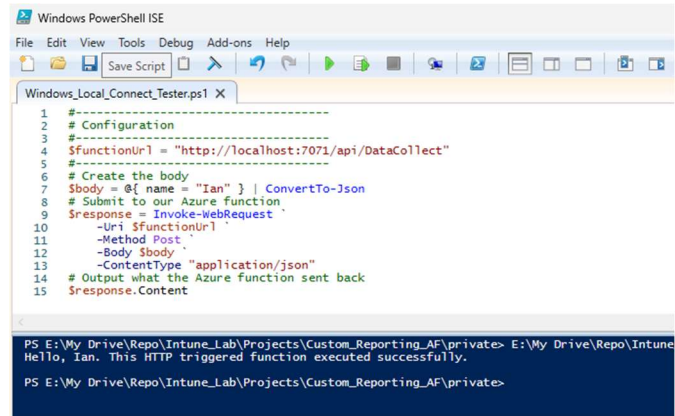
Hit Ctrl-C in the Terminal to terminate the worker process and clear the terminal using the “cls” command.

Checking the “Problems” tab shows an issue importing Azure.Functions module. This can be ignored.

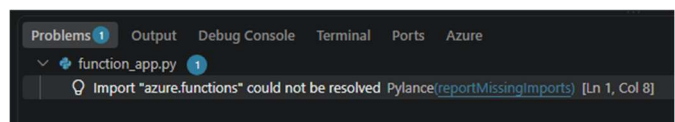


```
PS E:\My Drive\Repo\Intune_Lab\Projects\Custom_Reporting_AF\private\Data_Collect> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
My Drive\Repo\Intune_Lab\Projects\Custom_Reporting_AF\private\Data_Collect> .\venv\Scripts\activate.ps1
(.venv) PS E:\My Drive\Repo\Intune_Lab\Projects\Custom_Reporting_AF\private\Data_Collect> func start

Azure Functions Core Tools
Core Tools Version: 4.12.0-beta[2020-08-17T18:36:58.116Z] Worker process started and initialized.
```



```
1 #-----
2 # Configuration
3 #-----
4 $functionUrl = "http://localhost:7071/api/DataCollect"
5 #-----
6 # Create the body
7 $body = @{ name = "Ian" } | ConvertTo-Json
8 # Submit to our Azure function
9 $response = Invoke-WebRequest `
10     -Uri $functionUrl `
11     -Method Post `
12     -Body $body `
13     -ContentType "application/json"
14 # Output what the Azure function sent back
15 $response.Content
```



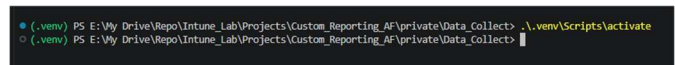
```
function_app.py 1
Import "azure.functions" could not be resolved. Pylance(reportMissingImports) [Ln 1, Col 8]
```

Deploy to Azure and Test with mTLS

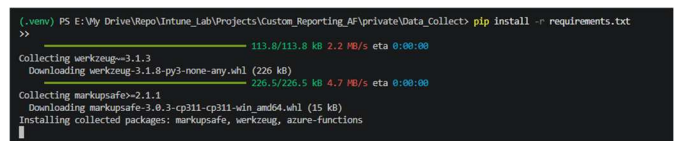
In the Terminal, type “.\venv\Scripts\activate” and hit Enter.

In my case, it was already activated.

In the Terminal, type “pip install -r requirements.txt” and hit Enter.



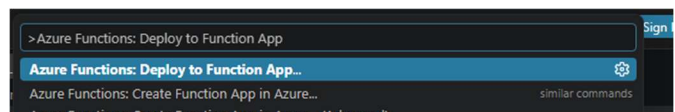
```
(.venv) PS E:\My Drive\Repo\Intune_Lab\Projects\Custom_Reporting_AF\private\Data_Collect> .\venv\Scripts\activate
```



```
(.venv) PS E:\My Drive\Repo\Intune_Lab\Projects\Custom_Reporting_AF\private\Data_Collect> pip install -r requirements.txt
>>
Collecting werkzeug==3.1.3
  Downloading werkzeug-3.1.3-py3-none-any.whl (226 kB)
Collecting markupsafe==2.1.1
  Downloading markupsafe-2.1.1-cp311-cp311-win_umd64.whl (15 kB)
Installing collected packages: markupsafe, werkzeug, azure-functions
```

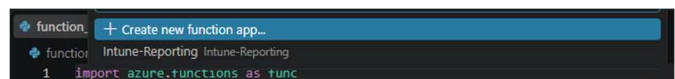
Hit “F1” and enter “Azure: Sign In”. Hit Enter and complete the sign in process.

Hit “F1” again and type “Azure Functions: Deploy to Function App” and hit Enter.



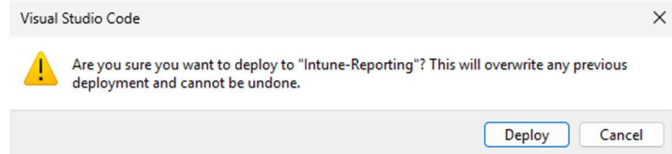
```
>Azure Functions: Deploy to Function App
Azure Functions: Deploy to Function App...
Azure Functions: Create Function App in Azure...
Azure Functions: Create Function App in Azure... (Advanced)
```

Select the Function App you created.

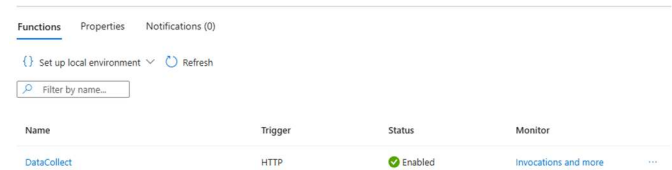


```
function. + Create new function app...
function. Intune-Reporting Intune-Reporting
1 import azure.functions as func
```

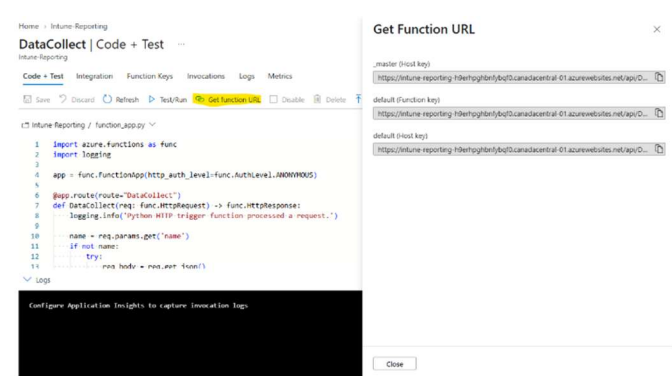
Click the “Deploy” button in the popup dialog.



You should now see the Azure Function listed in the Overview when you select the “Functions” tab.

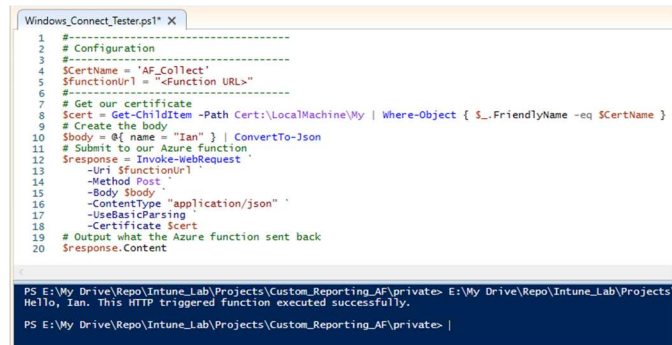


Click on the function to open it. Click the “Get Function URL” link.



In this case, all the URL's are the same, so copy and record the URL.

I have another Powershell script
“Windows_Azure_Connect_Tester.ps1”
that can be used to verify the Azure
Function that was deployed works with the
certificate.



Azure function hardening (Certificate validation)

Update Python script and Requirements

With the Azure Function project open in Visual Studio Code, replace the contents of “function_app.py” with the contents from the “cert_hardened_function_app.py” file.

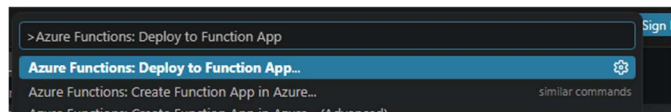
```
function_app.py 2 X
function_app.py > datacollect
1 import logging
2 import base64
3
4 import azure.functions as func
5 from cryptography import x509
6 from cryptography.hazmat.primitives import hashes
7 from cryptography.hazmat.primitives.asymmetric import padding
8 from cryptography.hazmat.backends import default_backend
9
10 from azure.identity import DefaultAzureCredential
11 from azure.keyvault.secrets import SecretClient
12
13 TRUSTED_ISSUER_CN = "IntuneLab_ROOT_CA2"
14 KEYVAULT_NAME = "kv-intune-ianbaxter"
15 CA_SECRET_NAME = "IntuneLab-ROOT-CA2"
16
17 # Cache CA cert in memory so we don't hit Key Vault on every request
18 _ca_cert = None
19
20
21 def _get_ca_cert() -> x509.Certificate:
```

The Requirements.txt file also needs to be updated to:

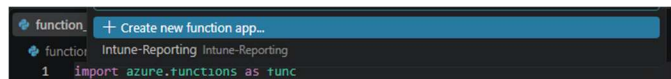
```
requirements.txt
1 # Uncomment to enable Azure Monitor OpenTelemetry
2 # Ref: aka.ms/functions-azure-monitor-python
3 # azure-monitor-opentelemetry
4
5 azure-functions
6 cryptography>=41.0.0,<42.0.0
7 azure-identity
8 azure-keyvault-secrets
```

Hit “F1” and enter “Azure: Sign In”. Hit Enter and complete the sign in process.

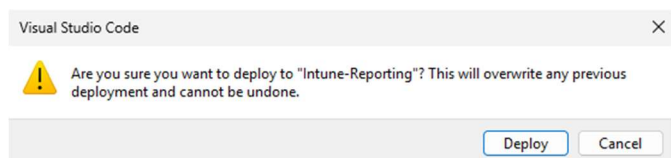
Hit “F1” again and type “Azure Functions: Deploy to Function App” and hit Enter.



Select the Function App you created.



Click the “Deploy” button in the popup dialog.



Check Azure Function Authentication Settings

Check the Authentication settings for the Function App. If you see this, then you will need to re-enable Anonymous access. You can verify this by checking the Azure Function URL.



Add an identity provider

Choose an identity provider to manage the user identities and authentication flow for your application. Providers include Microsoft, Facebook, Google, and Twitter. [Learn more about identity providers](#)

[Add identity provider](#)

You need to create a dummy managed identity. Click “Add Managed Identity.”

Select “Microsoft” as the “Service Provider”.

Set the “Secret Expiration” to 180 days.

Click “Add”.

Home > Intune-Reporting | Authentication

Add an identity provider

Basics Permissions

Identity provider *

Microsoft

Choose a tenant for your application and its users

A tenant contains applications and a directory for user accounts. Choose a tenant based on whether it's configured for workforce users (employees and business guests) or external users. [Learn more](#)

☒ Workforce configuration (current tenant)

Manage employees and business guests

☐ External configuration

Manage external users

App registration

An app registration associates your identity provider with your app. Enter the app registration information here, or go to your provider to create a new one. [Learn more](#)

App registration type *

☒ Create new app registration

☐ Pick an existing app registration in this directory

☐ Provide the details of an existing app registration

Name *

Intune-Reporting

Client secret expiration *

Recommended: 180 days

Supported account types *

☒ Current tenant - Single tenant

[Add](#)

[< Previous](#)

[Next: Permissions >](#)

Now you can edit the authentication settings.

Edit authentication settings

Requiring authentication ensures that requests to your app include information about the caller, but your app may still need to make additional authorization decisions to control access. If unauthenticated requests are allowed, any client can call the app and your code will need to handle both authentication and authorization. [Learn more](#)

App Service authentication *

☒ Enabled

☐ Disabled

Restrict access *

☐ Require authentication

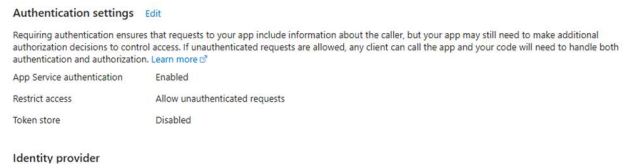
☒ Allow unauthenticated access

Token store

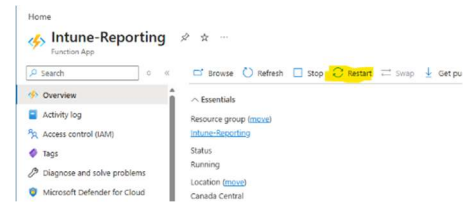
☐

Allowed external redirect URLs

Then I checked the authentication settings again:



Restart the Function App.



Test the updated azure function

Logging DCR Issues

Update Inventory Scripts for endpoint URL

Deploy Inventory Scripts